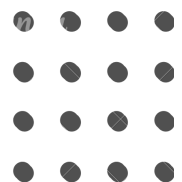




VMES Grinds

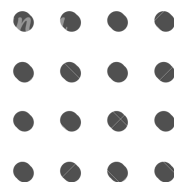




VMES Grinds

Pilot Program - Week 3 - Day 1

Sliding window, Two pointers





925. Long Pressed Name





925. Long Pressed Name

Easy

Topics

Companies

Your friend is typing his `name` into a keyboard. Sometimes, when typing a character `c`, the key might get *long pressed*, and the character will be typed 1 or more times.

You examine the `typed` characters of the keyboard. Return `True` if it is possible that it was your friends name, with some characters (possibly none) being long pressed.

Example 1:

Input: `name = "alex", typed = "aaleex"`

Output: `true`

Explanation: 'a' and 'e' in 'alex' were long pressed.

Example 2:

Input: `name = "saeed", typed = "ssaaedd"`

Output: `false`

Explanation: 'e' must have been pressed twice, but it was not in the typed output.

Constraints:

- `1 <= name.length, typed.length <= 1000`
- `name` and `typed` consist of only lowercase English letters.



Explanation





Thought Process

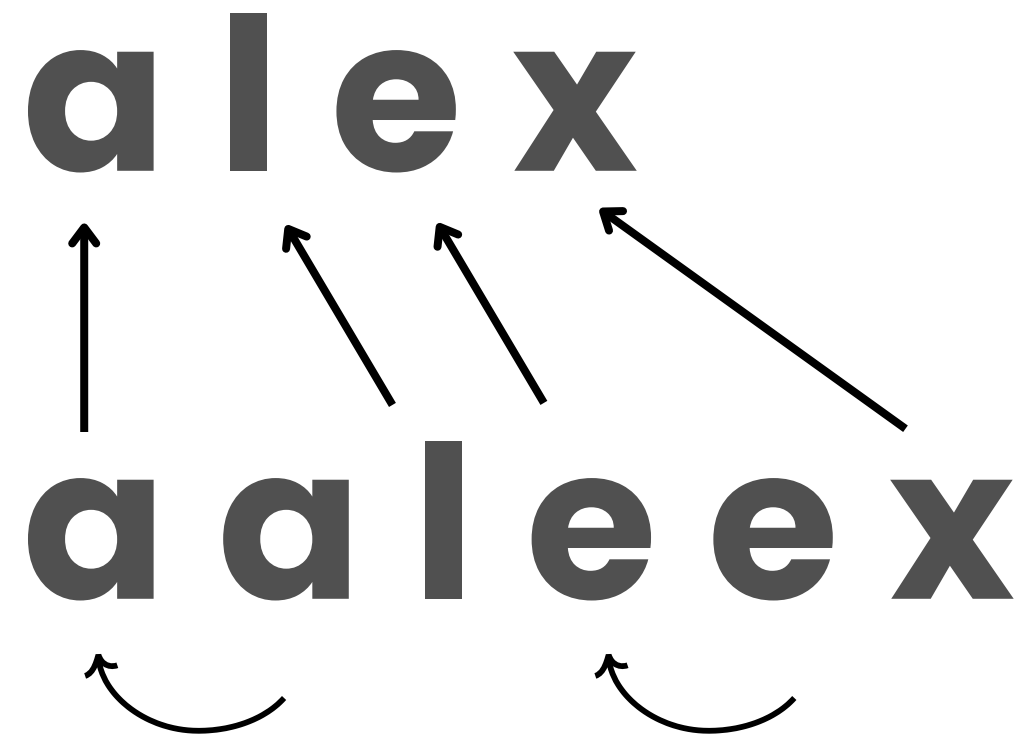
- **Use two pointers (one in each string)**
- **Linear iteration through “typed” string**
- **Check conditions**



Example:

name = "alex"

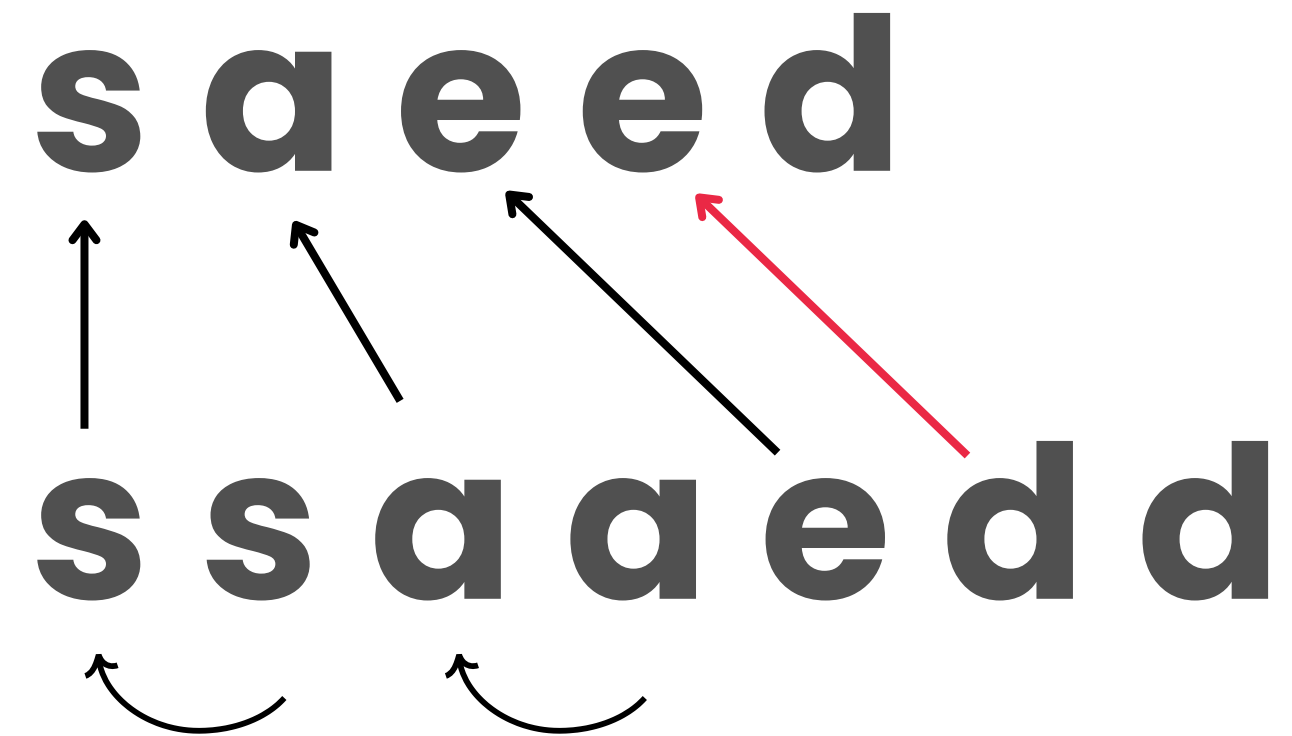
typed = "aaleex"



Example:

name = "saeed"

typed = "ssaaedd"





Code





```
class Solution:
    def isLongPressedName(self, name: str, typed: str) -> bool:
        i = 0
        j = 0

        if len(name) > len(typed):
            return False

        while j < len(typed):
            if i < len(name) and name[i] == typed[j]:
                i += 1
                j += 1
            elif i > 0 and name[i-1] == typed[j]:
                j += 1
            else:
                return False
        return i == len(name) and j == len(typed)
```



Optimized Code





```
class Solution:
    def isLongPressedName(self, name: str, typed: str) -> bool:
        i = j = 0

        while j < len(typed):
            if i < len(name) and name[i] == typed[j]:
                i += 1
                j += 1
            elif j > 0 and typed[j] == typed[j - 1]:
                j += 1
            else:
                return False

        return i == len(name)
```



Time and Space Complexity Analysis





```
class Solution:
    def isLongPressedName(self, name: str, typed: str) -> bool:
        i = j = 0    ← Space - O(1)

        while j < len(typed):    ← Time - O(n)
            if i < len(name) and name[i] == typed[j]:
                i += 1
                j += 1
            elif j > 0 and typed[j] == typed[j - 1]:
                j += 1
            else:
                return False

        return i == len(name)
```




2760. Longest Even Odd Subarray With Threshold





2760. Longest Even Odd Subarray With Threshold

Solved

Easy

Topics

Companies

Hint

You are given a **0-indexed** integer array `nums` and an integer `threshold`.

Find the length of the **longest subarray** of `nums` starting at index `l` and ending at index `r` ($0 \leq l \leq r < \text{nums.length}$) that satisfies the following conditions:

- `nums[l] % 2 == 0`
- For all indices `i` in the range `[l, r - 1]`, `nums[i] % 2 != nums[i + 1] % 2`
- For all indices `i` in the range `[l, r]`, `nums[i] <= threshold`

Return an integer denoting the length of the longest such subarray.

Note: A **subarray** is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [3,2,5,4]`, `threshold = 5`

Output: 3

Explanation: In this example, we can select the subarray that starts at `l = 1` and ends at `r = 3` $\Rightarrow$ `[2,5,4]`. This subarray satisfies the conditions. Hence, the answer is the length of the subarray, 3. We can show that 3 is the maximum possible achievable length.



Example 2:

Input: `nums = [1,2], threshold = 2`

Output: `1`

Explanation: In this example, we can select the subarray that starts at `l = 1` and ends at `r = 1` $\Rightarrow$ `[2]`.
It satisfies all the conditions and we can show that 1 is the maximum possible achievable length.

Example 3:

Input: `nums = [2,3,4,5], threshold = 4`

Output: `3`

Explanation: In this example, we can select the subarray that starts at `l = 0` and ends at `r = 2` $\Rightarrow$ `[2,3,4]`.
It satisfies all the conditions.
Hence, the answer is the length of the subarray, 3. We can show that 3 is the maximum possible achievable length.

Constraints:

- `1 <= nums.length <= 100`
- `1 <= nums[i] <= 100`
- `1 <= threshold <= 100`



Explanation





Thought Process

- **Linear iteration through array**
- **Check threshold**
- **Initialize even index**
- **Check Even Odd or not**



Example:

nums = [3,2,5,4], threshold = 5

3,[2,5,4]

- starts with even
- all $\leq$ threshold
- [E,O,E]

Example:

nums = [2,3,4,5], threshold = 4

[2,3,4],5

- starts with even
- all $\leq$ threshold
- [E,O,E]



Code





```
class Solution:
    def longestAlternatingSubarray(self, nums: list[int], threshold: int) -> int:
        left = 1000
        right = -1000
        res = 0

        for i in range(len(nums)):
            if nums[i] <= threshold:
                if left == 1000 and nums[i] % 2 == 0:
                    left = i
                    right = i
                if i < len(nums)-1 and nums[i+1] <= threshold and nums[i] % 2 != nums[i+1] % 2:
                    right = i+1
                else:
                    res = max(res, i-left+1)
                    left = 1000
                    right = -1000
                res = max(res, right-left+1)
            else:
                left = 1000
                right = -1000

        return res
```




Optimized Code





```
class Solution:
    def longestAlternatingSubarray(self, nums: list[int], threshold: int) -> int:
        left = -1
        res = 0

        for i in range(len(nums)):
            if nums[i] > threshold:
                left = -1
                continue

            if left == -1:
                if nums[i] % 2 == 0:
                    left = i
                else:
                    continue

            if i + 1 < len(nums) and nums[i+1] <= threshold and (nums[i] % 2 != nums[i+1] % 2):
                res = max(res, i - left + 2)
            else:
                res = max(res, i - left + 1)
                left = -1

        return res
```



Time and Space Complexity Analysis





```
class Solution:
    def longestAlternatingSubarray(self, nums: list[int], threshold: int) -> int:
        left = -1
        res = 0
        for i in range(len(nums)):
            if nums[i] > threshold:
                left = -1
                continue

            if left == -1:
                if nums[i] % 2 == 0:
                    left = i
                else:
                    continue

            if i + 1 < len(nums) and nums[i+1] <= threshold and (nums[i] % 2 != nums[i+1] % 2):
                res = max(res, i - left + 2)
            else:
                res = max(res, i - left + 1)
                left = -1

        return res
```

← Space - $O(1)$

← Time - $O(n)$



Key Takeaway





- **Edge cases are important.**
- **Consider Optimization**
 - **When solution is correct.**
- **Sometimes, unnecessary things lead to inefficiency.**
 - **Extra space and time complexity.**



VMES Grinds



Thank You !